Monday, March 10, 2025

LEC19 - Binary Search Trees

Passing Functions to Functions

- Use the callable type

```
def list_if(self, p: Callable[[Any], bool]) -> list[Any]:
      """ Return a list of all values in this tree that satisfy
predicate <p>

Parameter <p> is a function which takes in a single argument value and
checks some predicate on that value, returning True or False
      """
      if self._root is empty:
            return []
      elif self._subtrees == []:
            return [self._root] if p(self._root) else []
      else:
            result = []
            for subtree in self._subtrees:
                  if p(subtree):
                        result.append(subtree._root)
            return result
```

Idea

- If self._root is empty, return []
- If the node is a leaf, return [self._root] if p(self._root) else []
- Otherwise, recursively for each subtree in self._subtrees and construct the list of values that satisfy p

---

Tree Traversal

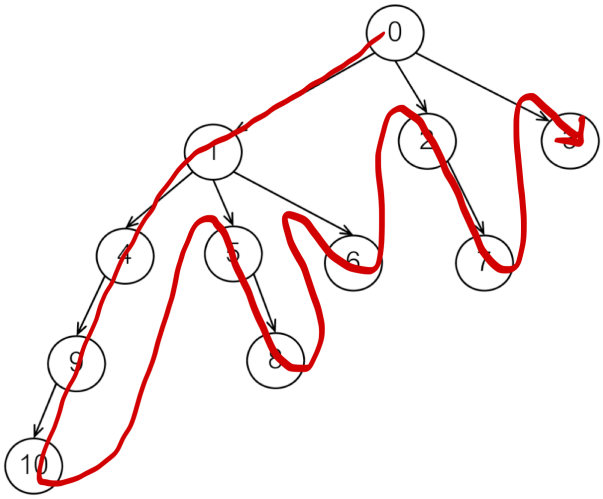
- So far, the functions and methods we have used for trees get information from all nodes in the tree
  - In a way, they already traverse the tree
- Sometimes the order in which elements are processed is important
  - Do we process the root of the tree before its children? (Prelevel)
  - Do we process along levels of depth in the tree? (Level)

Preorder Traversal

- Visit each tree node as follows
  - Do something with the current node
  - Visit its first subtree in preorder
  - Visit its second subtree in preorder

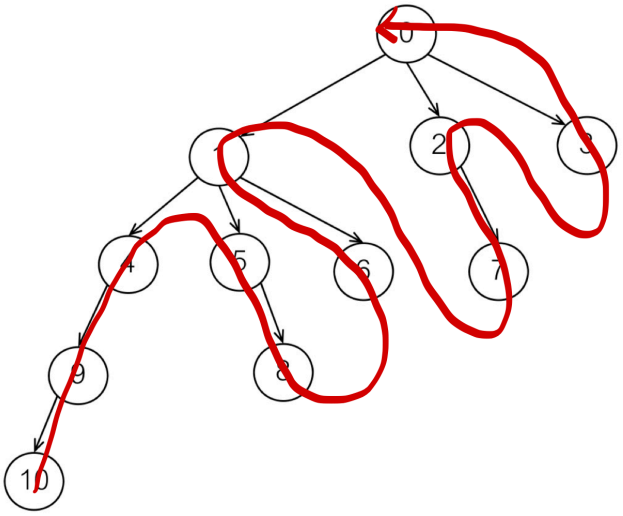
Monday, March 10, 2025

LEC19 - Binary Search Trees



Postorder Traversal

- Visit each tree node as follows
  - Visit its first subtree in postorder
  - Do something with the node



Level Order Traversal

- Visit each tree node as follows
  - Do something with the node
  - Visit all its children (first level of the tree) and act on the nodes
  - Visit all the childrens children (second level of the tree) and act on the nodes
  - Repeat until last node is visited

Monday, March 10, 2025

LEC19 - Binary Search Trees